
The password that stopped existing

Secrets management, learned the hard way

DEVOPS INTERNSHIP · WEEK 5



WEEK 1

The key in the public repo

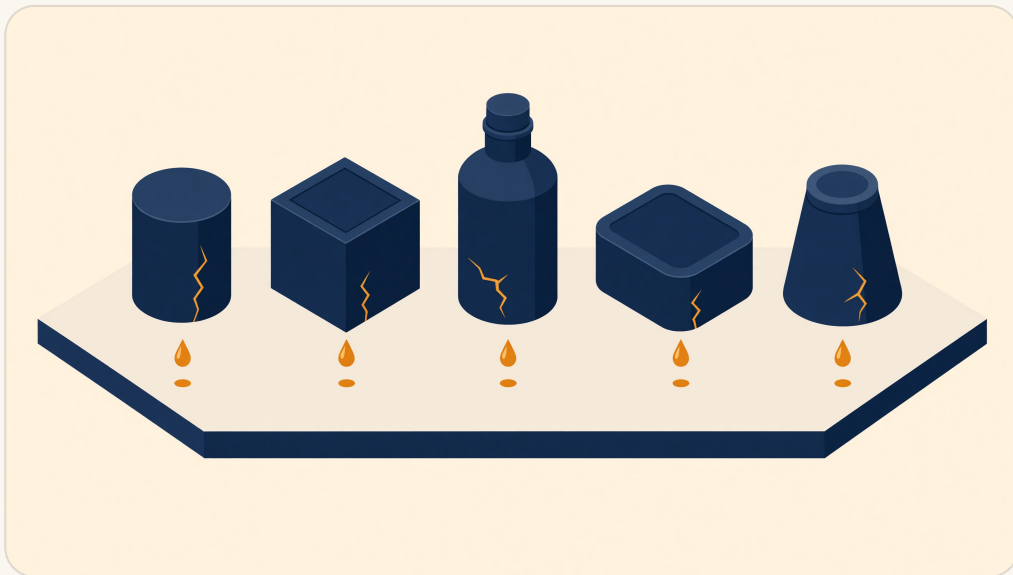
- Terraform credentials committed to GitHub
- AWS emailed me before I noticed
- Deleted the file — **the key was still in git history**

Time to detection: **hours**. Time to exploitation: **minutes**.



WHY IT HAPPENS

Five places secrets end up without anyone deciding



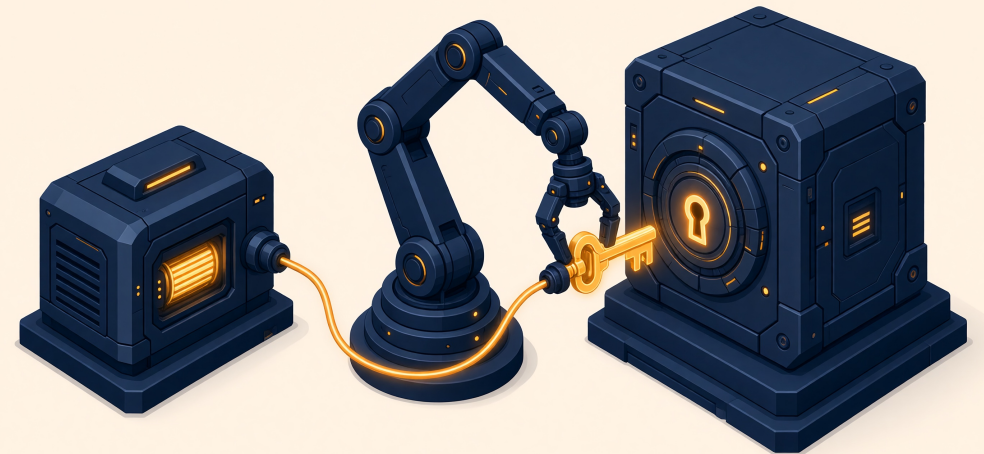
- 1 Git history — survives deletion
- 2 CI/CD logs — printed by a debug command
- 3 Terraform state — every attribute, in plain text
- 4 `.env` files — copied between machines
- 5 Container images — baked into a layer

STEP ONE

The secret moves to a managed store

- Terraform generates the password — nobody types it
- Stored in SSM Parameter Store as `SecureString`
- Ansible reads it at deploy time, `no_log: true`
- A custom module masks the value in its own output

The value never appears in a repository or in a log.



THE GAP NOBODY MENTIONS

Terraform state holds everything



```
resource "random_password" "db" {  
  length = 32  
}
```

- `sensitive = true` hides it from **command output**
- It is still **plain text** in the state file
- State lives in S3 — anyone with bucket access reads the password

STEP TWO

The password nobody created

```
manage_master_user_password = true
```

- RDS generates the password itself
- Stores it in Secrets Manager, encrypted with our own key
- Rotates it on a schedule
- Terraform only ever sees the **ARN**

Not in the code. Not in the variables. **Not in the state.**



STEP THREE

No password at all



```
iam_database_authentication_enabled = true
```

- The application asks AWS for a token
- Token is valid for **15 minutes**
- Revoked by removing a permission, not by changing a password
- Every connection appears in CloudTrail

ENCRYPTION HAS A SECOND LOCK

Two things that look identical and are not

- Permission to **use a service** — IAM policy
- Permission to **use the key it encrypts with** — key policy

Found before deployment, in this project:
— log groups would not have been created
— security alerts would have failed **silently**



DETECT WHAT YOU CANNOT PREVENT

The alert path, proven end to end



API call → CloudTrail → EventBridge → SNS → email

- Root sign-in
- Security group change
- Audit trail stopped
- IAM user created — in a role-only account

The email, one minute after the change

Who, from where, with which tool, at what time — and over which TLS version

TAKEAWAYS

Five things I would tell myself in week one

- 1 Rotate the key, not the file — deletion is not revocation
- 2 `sensitive` hides output, not storage
- 3 The safest secret is the one you never hold
- 4 Short-lived tokens beat long-lived passwords
- 5 Test the alert, or you do not have one

